

Signal

Audio Manager for Unity

*A flexible, performant, and data-driven audio solution
designed for a fast workflow and professional results*

Version 1.0
Unity 2022.3 LTS or newer
Date February 24, 2026
Support chspcontato@gmail.com
Author carlosbobao, Resenha Studio

Contents

Introduction	2
1 Core Features	2
2 Installation	3
2.1 Option A — Unity Package Manager via Tarball (Recommended)	3
2.2 Option B — Unity Package (Drag-and-Drop)	3
3 Quick Start	3
3.1 Step 1 — Generate the Initial Setup	3
3.2 Step 2 — Prepare Your Audio Files	5
3.3 Step 3 — Populate the Audio Database	6
3.4 Step 4 — Generate the AudioKeys File	6
3.5 Step 5 — Play Your First Sound	7
4 Using the API	7
4.1 Required Namespaces	7
4.2 Sound Effects	7
4.3 Delayed Sound Effects	8
4.4 UI Sounds	8
4.5 Music	8
4.6 Looping Sound Instances	8
4.7 Global Volume Control (Settings Menu)	9
4.8 Inspector-Driven Sounds	10
4.9 Setting Up Ambient Zones	10
5 API Reference	10
6 Troubleshooting	12
6.1 Compilation Errors After Import	12
6.2 Sound Is Not Playing	12
6.3 Other Common Issues	13
7 License	14

Introduction

Managing audio in Unity can quickly become a maintenance problem. Hard-coded string references, scattered `AudioSource` components, and disorganized manager scripts slow down development and introduce silent bugs that are only caught during playtesting.

Signal Audio Manager solves these problems with a robust, data-driven framework built on Scriptable Objects. It provides a centralized database for all of your game's audio — music, sound effects, UI sounds, and ambient tracks — paired with a clean static API that can be called from any script, in any scene, with **no scene configuration required**.

Designed to be approachable for beginners while offering the architecture that experienced developers expect, Signal scales cleanly from a 48-hour game jam to a full commercial release.

1 Core Features

Data-Driven Architecture

All sounds are configured through the `SoundManagerSO` Scriptable Object. No hard-coded paths. No scattered references across scenes.

Decoupled Static API

The `Signal` class exposes a fully static API backed by an event bus (`AudioEventChannel`). Play any sound from anywhere in your codebase without holding a reference to the manager.

Automated Lazy Loading

Signal auto-instantiates itself on the first API call via `EnsureManagerExists()` — from any script, in any scene, including additive loading. **You never need to drag a prefab into a scene.**

UI Toolkit Dashboard

A custom Editor window built with Unity's modern UI Toolkit. Manage your entire SFX, Music, and UI library from a single responsive hub, with live visual error warnings that catch broken references before you press Play.

Typo-Proof API — Code Generation

One click generates a static `AudioKeys.cs` class where every audio ID is a typed constant. Full IDE autocomplete. The compiler catches renamed or missing entries immediately.

Bulk Audio Import

Select a source folder, define a naming prefix, and Signal imports every clip automatically — correctly named, categorized, and ready to use.

Batch File Renamer

Downloaded audio files rarely come with the right naming convention. The built-in Batch Renamer adds a prefix (e.g., `sfx_` or `ost_`) to hundreds of files in one click — directly inside the Unity Editor, before importing.

Automatic Variation

Assign multiple `AudioClips` to a single ID with independent Min/Max sliders for

volume and pitch. Footsteps, impacts, and UI sounds will never feel repetitive.

⚡ Smart Object Pooling

SFX and UI audio sources are managed through high-performance pools (20 SFX + 10 UI sources by default) that produce **zero GC allocations** during gameplay. Essential for mobile and frame-rate-sensitive projects.

🔊 Dedicated UI Audio Channel

UI sounds run on a separate mixer group using **unscaled time**. Menu sounds play correctly even when `Time.timeScale = 0`.

🏠 Dynamic Ambient System

Create crossfading ambient soundscapes triggered by player location using the `AmbientTrigger` component and `AmbientTypeSO` Scriptable Objects.

🔁 Looping Sound Instance Control

`Signal.PlaySFXLoop()` returns a unique `long` instance ID, letting you stop specific looping sounds manually — perfect for machine hums, alarms, or any persistent audio.

⚙️ Global Volume Controls with Persistence

Dedicated mixer volume controls for Master, Music, and SFX channels. All settings are automatically saved via `PlayerPrefs` and restored on the next session — ready to wire directly to your Settings Menu sliders.

2 Installation

2.1 Option A — Unity Package Manager via Tarball (Recommended)

1. In Unity, open *Window > Package Manager*.
2. Click the + button (top-left) and select **Add package from tarball...**
3. Navigate to the downloaded `signal-audio-manager-unity_1.0.0.tgz` file and confirm.

2.2 Option B — Unity Package (Drag-and-Drop)

1. Download the `signal-audio_FastSetupPackage.unitypackage` file.
2. Drag and drop it into your Unity project window.
3. Click **Import All** when prompted.

3 Quick Start

This section walks you through the complete setup from a blank project to your first sound playing in under five minutes.

3.1 Step 1 — Generate the Initial Setup

(Done once per project)

1. Open *Window > Signal Audio > Dashboard*.

2. Click **Generate Initial Setup** and choose a destination folder.

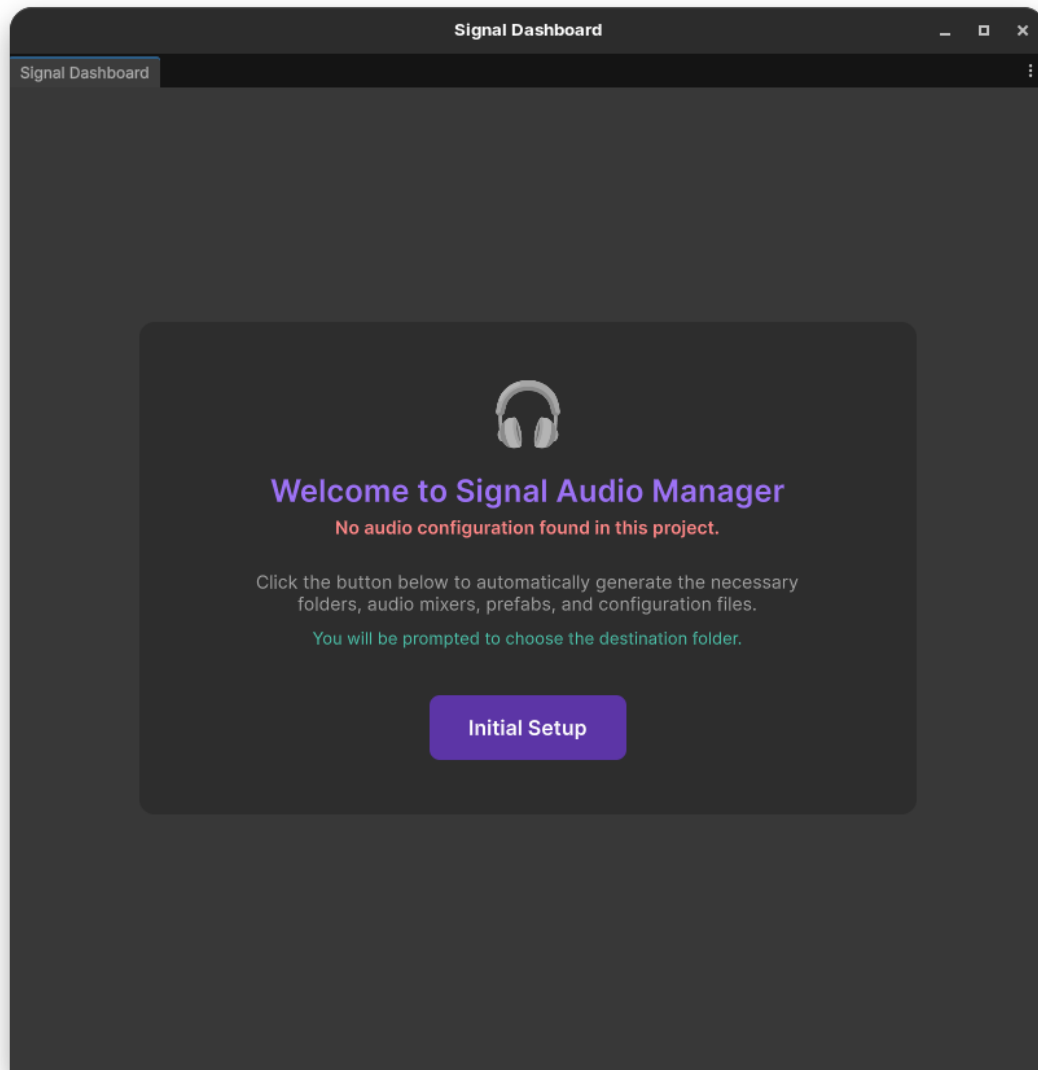


Figure 1: The Signal Dashboard — Generate Initial Setup

Signal will automatically create:

- **SoundManagerSO** — the central Scriptable Object that holds all configuration
- The Audio Mixer with Master, SFX, Music, and UI groups pre-configured
- The SFX pool Prefab (placed in **Resources/** and managed internally)

Signal uses Automated Lazy Loading powered by **EnsureManagerExists()**. The manager prefab is instantiated automatically the first time any **Signal.*** method is called, and is kept alive across scenes via **DontDestroyOnLoad**. You never need to place anything in a scene manually.

3.2 Step 2 — Prepare Your Audio Files

If your downloaded audio files do not yet follow a naming convention, rename them before importing.

1. In the Dashboard, open the **Import Tools** tab.
2. Select the folder containing your raw audio files and set the desired prefix (e.g., `sfx_` or `ost_`). Click **Rename All** — every file in the folder is renamed instantly.
3. Under the **Batch Renamer Tool**, select the source folder and click **Import SFX** or **Import Music**. Signal will scan the folder and load every compatible audio file into the corresponding library automatically.

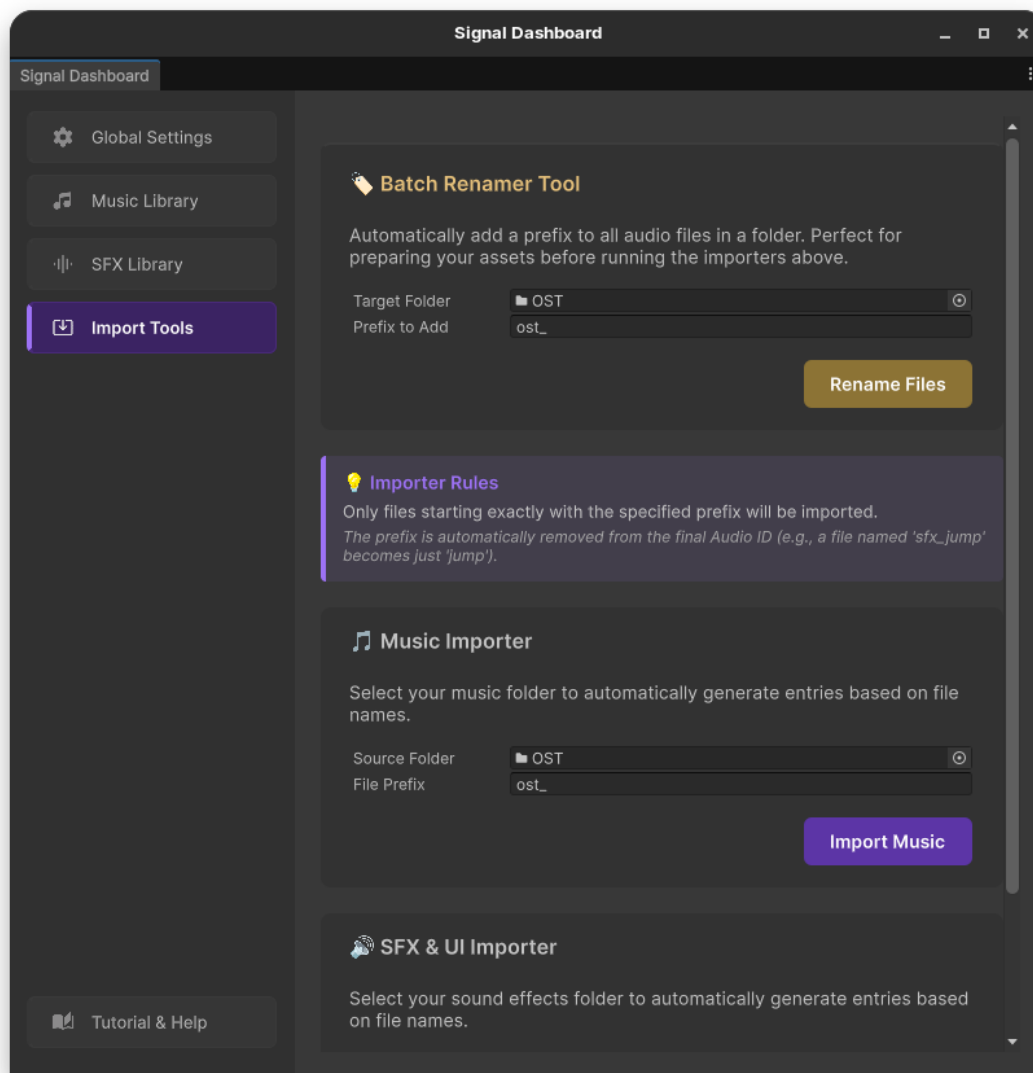


Figure 2: The SFX & UI Audio Management tab — AudioEntry with Min/Max Volume and Pitch sliders

3.3 Step 3 — Populate the Audio Database

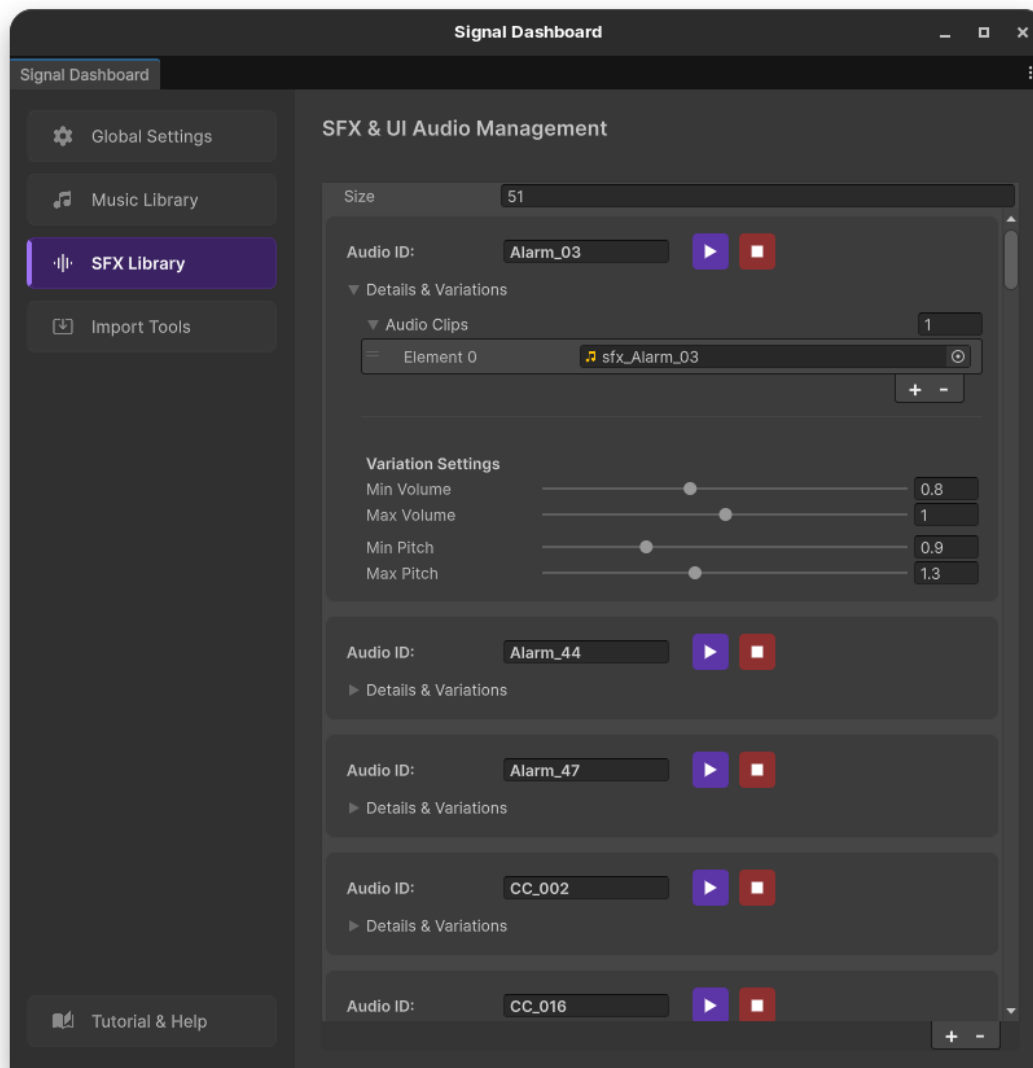


Figure 3: The Import Tools tab — Bulk Audio Import panel

1. In the Dashboard, open the **SFX Library** or **Music** tab.
2. Click any entry to expand it. Edit the **AudioID** to rename it, and adjust the **Min/Max Volume** and **Min/Max Pitch** sliders to control playback variation.
3. To add variation, assign multiple **AudioClips** to the same entry — Signal will pick one at random each time it plays.
4. Click the + button to create a new entry manually, or use **Bulk Import** to populate the library from a folder automatically.

3.4 Step 4 — Generate the AudioKeys File

1. In the Dashboard, go to the **Global Settings** tab.
2. Click **Generate AudioKeys**.

A static `AudioKeys` class is created with nested classes `AudioKeys.SFX`, `AudioKeys.Music`, and `AudioKeys.UI` — each containing every ID in your database as a typed constant. Your IDE will now autocomplete all audio calls.

3.5 Step 5 — Play Your First Sound

Call the API from any script. No scene setup required.

```
using SignalAudioManagerUnity.Communication;
using SignalAudioManagerUnity;
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    private void Jump()
    {
        // Full IDE autocomplete. Compiler catches typos and renamed IDs.
        Signal.PlaySFX(AudioKeys.SFX.player_jump, transform.position);
    }
}
```

Listing 1: Playing your first sound from any script in any scene

No `GetComponent`. No scene objects. No per-scene configuration. `Signal` instantiates and configures itself on first call and persists across all scene loads.

4 Using the API

4.1 Required Namespaces

```
using SignalAudioManagerUnity.Communication; // Signal static class
using SignalAudioManagerUnity;               // AudioKeys class
```

Listing 2: Add these using directives to any script that calls `Signal`

4.2 Sound Effects

All three `PlaySFX` overloads accept optional `volumeMultiplier` and `pitchMultiplier` parameters. These are multiplied on top of each `AudioEntry`'s configured Min/Max ranges.

```
// 2D SFX    no spatial position
Signal.PlaySFX(AudioKeys.SFX.coin_pickup);

// 3D SFX at a fixed world position
Signal.PlaySFX(AudioKeys.SFX.explosion, transform.position);

// 3D SFX attached to a Transform sound follows the object
Signal.PlaySFX(AudioKeys.SFX.footstep, transform);

// Optional runtime override of volume and pitch
Signal.PlaySFX(AudioKeys.SFX.impact, transform.position,
    volumeMultiplier: 0.6f, pitchMultiplier: 1.15f);
```

Listing 3: SFX playback — three spatial overloads

4.3 Delayed Sound Effects

```
// Play a sound 0.5 seconds after the call
Signal.PlaySFXDelayed(AudioKeys.SFX.explosion_rumble, delayInSeconds: 0.5f);

// With optional volume override
Signal.PlaySFXDelayed(AudioKeys.SFX.debris, delayInSeconds: 0.3f,
    volumeMultiplier: 0.8f);
```

Listing 4: Delayed SFX — perfect for syncing with animations or particles

4.4 UI Sounds

UI sounds are routed through a dedicated audio channel that uses **unscaled time**. They play correctly even when `Time.timeScale = 0`.

```
Signal.PlayUI(AudioKeys.UI.button_click);
Signal.PlayUI(AudioKeys.UI.menu_open);
```

Listing 5: UI sound playback

⚠ Warning

Never use `Signal.PlaySFX()` for interface sounds. Only `PlayUI()` routes audio through the unscaled time channel. Using `PlaySFX` for UI will cause those sounds to go silent whenever `Time.timeScale = 0`.

4.5 Music

```
// Play with default 2-second crossfade
Signal.PlayMusic(AudioKeys.Music.main_theme);

// Play with custom crossfade duration and target volume
Signal.PlayMusic(AudioKeys.Music.boss_theme, fadeDuration: 1.5f,
    targetVolume: 0.9f);

// Stop current music with fade-out
Signal.StopMusic();

// Pause and resume
Signal.PauseMusic();
Signal.ResumeMusic();
```

Listing 6: Music control

4.6 Looping Sound Instances

Use `PlaySFXLoop()` for persistent sounds that need to be stopped on demand. The method returns a unique long ID identifying that exact instance.

```
using SignalAudioManagerUnity.Communication;
using SignalAudioManagerUnity;
using UnityEngine;

public class Machine : MonoBehaviour
{
    // Store the ID as a class field never a local variable
    private long _humId = 0;
```

```

public void Activate()
{
    _humId = Signal.PlaySFXLoop(AudioKeys.SFX.machine_hum, transform);
}

public void Deactivate()
{
    Signal.StopInstance(_humId);
    _humId = 0;
}

private void OnDestroy()
{
    // Always clean up to avoid orphaned audio sources
    if (_humId != 0)
        Signal.StopInstance(_humId);
}
}

```

Listing 7: Full lifecycle of a looping sound instance

4.7 Global Volume Control (Settings Menu)

Signal exposes dedicated volume controls for each mixer channel, designed to be wired directly to UI sliders in a Settings Menu. All values are **normalized** (0.0 = muted, 1.0 = max volume). Values are automatically converted to decibels internally and **persisted to PlayerPrefs** — so the player's preferences are restored every session with no extra code.

```

using SignalAudioManagerUnity.Communication;
using UnityEngine;
using UnityEngine.UI;

public class SettingsMenu : MonoBehaviour
{
    [SerializeField] private Slider masterSlider;
    [SerializeField] private Slider musicSlider;
    [SerializeField] private Slider sfxSlider;

    private void Start()
    {
        // Restore saved values when the menu opens (range: 0.0 to 1.0)
        masterSlider.value = PlayerPrefs.GetFloat("MasterVolume", 1f);
        musicSlider.value = PlayerPrefs.GetFloat("MusicVolume", 1f);
        sfxSlider.value = PlayerPrefs.GetFloat("SFXVolume", 1f);
    }

    // Wire each of these to the slider's OnValueChanged event in the Inspector
    public void OnMasterChanged(float value) => Signal.SetMasterVolume(value);
    public void OnMusicChanged(float value) => Signal.SetMusicVolume(value);
    public void OnSFXChanged(float value) => Signal.SetSFXVolume(value);
}

```

Listing 8: Wiring volume sliders in a Settings Menu

```

// Change volume from 0.0 (Muted) to 1.0 (Max Volume)
Signal.SetMasterVolume(0.8f);
Signal.SetMusicVolume(0.5f);

```

```
Signal.SetSFXVolume(1.0f);  
Signal.SetUIVolume(1.0f);
```

Listing 9: Standalone volume calls — can be used anywhere

Signal converts normalized values to decibels using `Mathf.Log10(volume) * 20f` before applying them to the Audio Mixer. The `PlayerPrefs` keys used for persistence match the parameter names in your `SoundManagerSO`: "MasterVolume", "MusicVolume", "SFXVolume", and "UIVolume".

4.8 Inspector-Driven Sounds

For event-driven sounds (buttons, doors, collectibles), Signal includes ready-to-use components that require no code at all.

- **PlaySFX component** — Add to any `GameObject`. Set the `sfxID` field. Toggle **Play At Object Position** to enable 3D spatialization. Wire its public `Play()` method to any Unity Event (e.g., Button's `OnClick()`), or enable **Play On Start**.
- **PlayMusic component** — Same pattern. Set `musicID` and `fadeDuration`. Enable **Play On Start** or wire `Play()` to any event.
- **AmbientTrigger component** — Requires a Collider (automatically set to `Is Trigger`). Assign an `AmbientTypeSO` to **Ambient Type**. Configure **Trigger On Enter**, **Trigger On Exit**, and optionally a separate **Exit Ambient Type** for the outgoing zone. Set **Player Tag** to match your player object's tag (default: "Player").

4.9 Setting Up Ambient Zones

1. Create Ambient Type assets via *Create > Signal Audio Manager > Ambient Type* (e.g., *Forest*, *Cave*, *City*).
2. Create an empty `GameObject` with any Collider. The **AmbientTrigger** component will automatically enable `Is Trigger`.
3. Add the **AmbientTrigger** component. Assign the `AmbientTypeSO` and configure enter/exit behaviour.
4. On the **AmbientSoundManager** component (created automatically during Initial Setup), expand the **Ambient Configs** list. For each entry, assign an `AmbientTypeSO`, the `AudioID(s)` to play, and the **Fade In/Out Duration**.

`AmbientSoundConfig` supports a list of `audioIDs` per zone — useful for layered ambient tracks. The first ID in the list is used as the primary track. You can also call `AmbientSoundManager.Instance.ChangeAmbient(myAmbientType)` directly from code.

5 API Reference

All methods are static members of the `Signal` class (`SignalAudioManagerUnity.Communication` namespace).

Sound Effects & UI

Method Signature	Description
<code>Signal.PlaySFX(string id)</code>	2D SFX, no spatial position.
<code>Signal.PlaySFX(string id, Vector3 pos, float vol, float pitch)</code>	3D SFX at a fixed world position.
<code>Signal.PlaySFX(string id, Transform t, float vol, float pitch)</code>	3D SFX attached to a Transform. Sound follows the object.
<code>Signal.PlaySFXDelayed(string id, float delay, float vol)</code>	SFX played after a delay in seconds.
<code>Signal.PlaySFXLoop(string id, Transform t, float vol)</code>	Looping 3D SFX. Returns a <code>long</code> instance ID.
<code>Signal.StopInstance(long id)</code>	Stops the looping instance identified by its ID.
<code>Signal.PlayUI(string id, float vol)</code>	2D SFX on the dedicated unscaled UI channel.

Music

Method Signature	Description
<code>Signal.PlayMusic(string id, float fade, float vol)</code>	Plays a music track with crossfade. Fade defaults to 2.0s.
<code>Signal.StopMusic()</code>	Stops current music with a fade-out.
<code>Signal.PauseMusic()</code>	Pauses the current music track.
<code>Signal.ResumeMusic()</code>	Resumes the paused music track.

Global Volume Control

Method Signature	Description
<code>Signal.SetMasterVolume(float v)</code>	Master mixer volume. v: 0.0 (mute) to 1.0 (max). Auto-saved to PlayerPrefs.
<code>Signal.SetMusicVolume(float v)</code>	Music mixer channel volume. Auto-saved.
<code>Signal.SetSFXVolume(float v)</code>	SFX mixer channel volume. Auto-saved.
<code>Signal.SetUIVolume(float v)</code>	UI mixer channel volume. Auto-saved.

All volume values are normalized (0.0–1.0). Signal converts them internally to decibels and persists them via **PlayerPrefs**. The saved values are automatically restored on the next play session — no extra code required.

6 Troubleshooting

6.1 Compilation Errors After Import

Warning

If you encounter compilation errors after importing, open `SignalAudioManagerUnity.Editor.asmdef` and verify it references `SignalAudioManagerUnity` (the Runtime asmdef). If the reference is missing, add it and click **Apply**.

6.2 Sound Is Not Playing

Open the Dashboard. The live error warnings will highlight any **AudioEntry** with a missing or empty **AudioClip** assignment immediately.

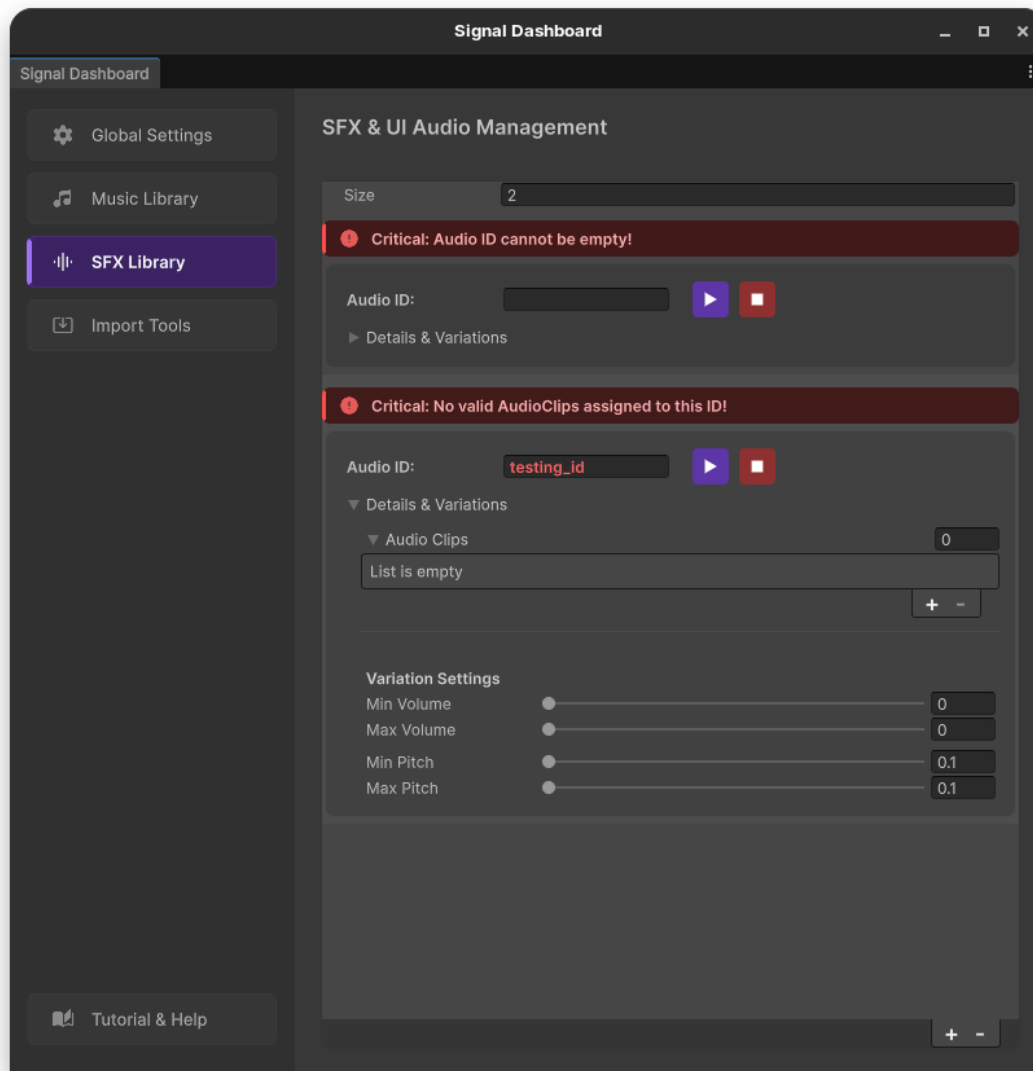


Figure 4: The Dashboard red error banner — shown when an AudioEntry has a missing clip

6.3 Other Common Issues

- **UI sounds are silent when the game is paused.**
Confirm you are calling `Signal.PlayUI()` for interface sounds and not `Signal.PlaySFX()`. Only `PlayUI()` routes through the unscaled time channel.
- **A looping sound won't stop.**
The return value of `Signal.PlaySFXLoop()` must be stored in a **class field**, never a local variable. Pass that exact value to `Signal.StopInstance()`. Also ensure `OnDestroy()` calls `StopInstance()` as a safety net.
- **AudioKeys.cs is not found by the compiler.**
Open the Dashboard, go to **Global Settings**, and click **Generate AudioKeys.cs**. If Unity does not detect the new file, right-click in the Project window and select **Refresh**.
- **Sound is not following a moving object.**

Use the **Transform** overload: `Signal.PlaySFX(id, transform)`. The **Vector3** overload places the audio source at a fixed position in the world.

- **Bulk import produced wrong or unexpected IDs.**

The **File Prefix** field is stripped from each filename to generate the **AudioID**. Verify the prefix in the **Import Tools** tab exactly matches the prefix on your files. Use the **Batch Renamer** first if your files are not yet correctly prefixed.

- **Volume slider has no effect.**

The **Audio Mixer** parameter names in **SoundManagerSO** must match the **Exposed Parameters** in your **Audio Mixer** asset exactly. The defaults are **MasterVolume**, **MusicVolume**, **SFXVolume**, and **UIVolume**.

- **Ambient zone is not triggering.**

Verify the player **GameObject** has the tag set in the **AmbientTrigger**'s **Player Tag** field (default: "Player"). Also confirm **Trigger On Enter** or **Trigger On Exit** is enabled, and that the **AmbientTypeSO** is assigned in **Ambient Configs** on the **AmbientSoundManager**.

7 License

Signal Audio Manager is provided under a **Commercial EULA**.

You are permitted to:

- Use and modify the asset in personal, educational, and commercial projects.
- Incorporate it into an unlimited number of compiled end products (games, applications).
- Publicly distribute compiled end products that include the asset.

You are not permitted to:

- Resell, sub-license, or redistribute the asset or any modified version as a standalone product.
- Share the source code with third parties outside of a compiled end product.
- Claim authorship or ownership of the asset.

The asset is provided **AS IS** without warranty of any kind. The author is not liable for damages arising from its use. See the **LICENSE.md** file included in the package for the full legal text.

For questions or support, contact chspcontato@gmail.com